Tracking of a High-Speed UAV Using an Error-State Extended Kalman Filter (ES-EKF)

Francisco Miranda, 20030411-3277
Maria Carolina Sebastião, 20031010 - T207

January 11, 2025

Abstract

This report addresses the implementation of an Error-State Extended Kalman Filter (ES-EKF) using quaternions to estimate the state (pose + IMU biases) of a 6-DoF UAV throughout a predetermined movement. With the dataset provided by UZH, a pseudo-ground-truth is first presented, followed by the real ground-truth data. The results, including the state error and the trajectory, are presented and the accuracy of the ES-EKF is discussed.

1 Introduction

Autonomous systems have become increasingly ubiquitous: cars, rockets, and drones, to name a few. They often are thoroughly equipped with intrinsic sensors, which allows dead-reckoning to be performed. However, this is prone to drifting, which will make the system diverge from its true state. For this reason, global measurements have to be included in the system. Clustering this into a formal procedure consists of implementing an estimation algorithm. Given the importance of the aforementioned systems, it's paramount to have reliable and time-efficient methods to estimate the system state online.

Numerous algorithms have been developed and implemented to address various types of problems. For tracking and global localization (*ie*, the environment map is known), variants of the Kalman Filter, particle filters and others are regularly used. For SLAM problems, one might resort to Graphical SLAM or FastSLAM, for instance.

In this paper, we propose to implement a variant of the Kalman Filter for a 6-DoF drone, solving a tracking problem. The UZH-FPV Drone Racing dataset [1] was central to developing this work, as it provided the IMU measurements and the ground-truth pose of the drone in several recorded motions. For simplicity, the 9th indoor dataset was used as it was regarded as a simpler motion.

1.1 Contribution

Drone tracking is a problem that has been drawing a lot of attention from the scientific community due to its growing importance in the surveillance and military fields [2]. There is a manifold of possible approaches to tackle this problem but it was concluded that one of the most famous methods was a variant of a Kalman Filter [2]: Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF), Invariant Extended Kalman Filter (IEKF) or Error-State Extended Kalman Filter (ES-EKF) are some of the options.

After a thorough investigation, the ES-EKF was chosen, which stemmed from its capacity to deal with nonlinear systems. As it will be explored in more detail, the ES-EKF has the advantage of working with an error state instead of the true state. Since the error state has more tendency to be linear than the true state, the linearization doesn't jeopardize the ES-EKF as much as other Kalman Filters [3].

Besides, several papers proceed to treat the UAV as a 2D system, simplifying the implementation (*e.g*, in [4]). In this project, we wanted to exploit the Kalman Filter for higher-dimensional systems, hence we encompassed every DoF. In order to avoid numerical stability problems, the analysis was conducted in the quaternion space .

1.2 Outline

This report will first provide an analysis of similar developments in this field and the respective estimation techniques will be briefly mentioned.

The implementation of the ES-EKF will then be outlined in detail. We'll start by explaining the ES-EKF and how it differs from the EKF. Since quaternions will be used and the UAV state is handled differently compared to the EKF, some equations differ from those covered in the course. Therefore,

some theoretical background will be presented, allowing the reader to fully go along with this report. This part will be divided into two: firstly we'll present the equations regarding the mean and then the equations regarding the covariance. Some personal decisions regarding the computational implementation will also be mentioned.

Afterwards, the structure of the performed simulations will be explained. To correctly evaluate the implementation, a well-organized workflow must be established, which led to the creation of a custom-made dataset. Once validated, the ground-truth data provided by UHV will be finally used.

Finally, the results will be presented, with the aim of discussing the influence of noise and outliers in the estimation algorithm and the complexities that real-life datasets entail when it comes to applying a Kalman filter.

2 Related Work

As previously stated, the estimation of a UAV state (whether it consists of its the pose or both the pose and map) has become highly important with the increase in the use of these type of tecnhologies. Consequently, there are numerous recent papers about this field.

In [5], an Uncertainty and Error-Aware Kalman Filter (UEAKF) is discussed. It starts by explaining the limitations of the traditional Kalman Filter and Particle Filter. Even though they're widely used in many applications, they are heavily influenced by high noise levels, especially when there is a lack of prior knowledge about the system's dynamics or when the environment is highly stochastic.

While Kalman Filter relies on fixed covariance matrices, based on the assumption that the noise is Gaussian, the Particle Filter can suffer from degradation in its accuracy when noise levels are strong. The latter handles better non-linearities but both can perform poorly in these conditions. Conversely, the UEAKF addresses these challenges by dynamically adjusting the covariance matrices. This error-aware mechanism is achieved based on the real-time uncertainties in the environment, even without prior knowledge regarding the noise distribution.

The authors thus show that this approach yields better results than the aforementioned ones in terms of Root Mean Square Error (RMSE).

In the study conducted by Pilté et al. [4], an innovative UKF is proposed to track an UAV pose using a 2D kinematic model. Firstly, the instability of the regular EKF, particularly under large initial errors and nonlinear dynamics, is addressed, which is followed by the introduction to the 2D kinematic model. It describes drone motion using intrinsic parameters, including curvature and angular velocity. This approach simplifies the measurement process, which relies on range and bearing coordinates, avoiding computationally expensive odometry data.

Afterwards, the paper introduces the IEKF. This filter still considers a prediction/propagation step and an update step. One of the main differences in comparison to the EKF for instance is the operation on Lie Groups. Besides, in a perfect theoretical setting, the linearizations do not depend on the predicted state. Given the kinematic model, this makes it more robust than the EKF, at least in what the prediction step is concerned. The same cannot be said about the update step, and that's why the authors couple the IEKF with the UKF to develop their method. The (left) UKF is then proposed, which includes the IEKF prediction step and the regular UKF update step.

Several experiments were conducted using real drone trajectory data and they showed that both IEKF and left-UKF outperform the EKF in root mean squared error (RMSE) for position and orientation estimates. Besides, the left-UKF provided marginally better results than the IEKF, especially for orientation precision.

In [6], Markovic et al. proposed an Error-State Extended Kalman Filter (ES-EKF) adapted for indoor UAV localization. It is shown as being more robust at the handling of sensor drift and calibration inaccuracies: by keeping error states small in magnitude, the method prevents issues like gimbal lock and maintains numerical stability, even during complex UAV maneuvers. This is possible because it predicts and updates the mean and covariance of the error state, rather than the state itself.

Experimental validation carried out in a controlled indoor setup showed that ES-EKF yields better results than individual sensor-based systems, demonstrating its accuracy in fusing diverse data contents for UAV localization.

Several papers focus on 3DoF kinematics, such as [4], thus simplifying the dynamics of the system. Conversely, this report (based on the KF foundations) tackles the challenges of nonlinear 6 DoF UAV tracking, in an attempt to better simulate real-world UAVs' dynamics. This is achieved through quaternion-based kinematics, which provides a more accurate representation of the system's dynamics.

For this reason, Solá's [3] and Markovic's et al.[6] works were central for this implementation, since they offered a complete overview of quaternion kinematics and ES-EKF considerations.

In conclusion, this project is partially based on previously done theoretical work but intends to apply to a 6 DoF UAV using a less common KF. It uses the UZH-FPV Drone Racing dataset to perform UAV tracking and presents some conclusions

regarding the ES-EFK algorithm.

3 My Method

As the ES-EKF has its differences in comparison to the EKF, an introductory overview is needed. Then the theoretical foundations will be presented, along with some decisions regarding the update step.

3.1 Overall structure

The ES-EKF consists of two main steps: the prediction and update steps. While the former predicts the mean state and the its covariance matrix, the latter takes the measurements and modifies the mean and covariance accordingly.

However, the ES-EKF differs from the EKF in one central aspect: it handles the error state rather than the state itself, which comes in handy as the error is much more prone to be linear than the state. Therefore, the state can be divided into an error component and a nominal component, leading to the creation of a new step: the injection and reset step, where we inject the error state in the nominal state and reset not only the former but also the covariance matrix. Figure 1 shows a concise representation of the ES-EKF (without the inject and reset step).

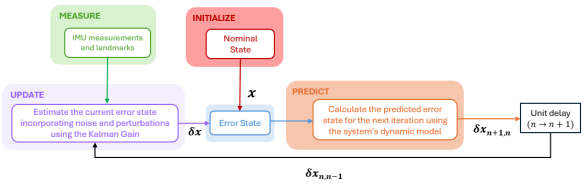


Figure 1: Diagram explaining the ES-EKF

3.2 Theoretical Concepts

3.2.1 Nominal and error states: predict and inject phases

The nominal state focuses on estimating the system's state x by integrating high-frequency IMU data while accounting for possible model inaccuracies. The nominal state is given by

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} & \mathbf{v} & \mathbf{q} & \mathbf{a}_b & \boldsymbol{\omega}_b & \mathbf{g} \end{bmatrix}^T \tag{1}$$

and consists of seven state elements: position, linear velocity, quaternion, linear acceleration bias, angular velocity bias and gravity, respectively.

The dynamic model was defined in a quaternion-based form. Considering the frames in which each state is defined, the equations for the nominal state are given by eq. 2 - 7, as described in [3].

$$\mathbf{p} \leftarrow \mathbf{p} + \mathbf{v}\Delta t + \frac{1}{2}\left(\mathbf{R}(\mathbf{a}_m - \mathbf{a}_b) + \mathbf{g}\right)\Delta t^2 \tag{2}$$

$$\mathbf{v} \leftarrow \mathbf{v} + \left(\mathbf{R}(\mathbf{a}_m - \mathbf{a}_b) + \mathbf{g}\right)\Delta t \tag{3}$$

$$\mathbf{q} \leftarrow \mathbf{q} \otimes \mathbf{q}\{(\boldsymbol{\omega}_m - \boldsymbol{\omega}_b)\,\Delta t\} \tag{4}$$

$$\mathbf{a}_b \leftarrow \mathbf{a}_b \tag{5}$$

$$\boldsymbol{\omega}_b \leftarrow \boldsymbol{\omega}_b \tag{6}$$

$$\mathbf{g} \leftarrow \mathbf{g} \tag{7}$$

Where $\bigotimes$ designates quaternion multiplication and $q\{\boldsymbol{\theta}\}$ is the quaternion associated to the rotation $\boldsymbol{\theta}$.

However, this approximation leads to the accumulation of errors over time. These are collected in the error state vector, which can be represented in its abbreviated form as:

$$\delta\mathbf{x} = \begin{bmatrix} \delta\mathbf{p} & \delta\mathbf{v} & \delta\mathbf{a}_b & \delta\boldsymbol{\omega}_b & \delta\mathbf{g} \end{bmatrix}^T, \tag{8}$$

The error state also has an associated motion model, presented in eq. 9 - 14. As the reader can note, the process noise is encompassed in the error state dynamics: $\boldsymbol{v_i}, \boldsymbol{\theta_i}, \boldsymbol{a_i}$ and $\boldsymbol{\omega_i}$ are random impulses determined by the process covariance matrix R_i.

$$\delta\boldsymbol{p} \leftarrow \delta\boldsymbol{p} + \delta\boldsymbol{v}\Delta t, \tag{9}$$

$$\delta\boldsymbol{v} \leftarrow \delta\boldsymbol{v} + \Big(- \boldsymbol{R}[\boldsymbol{a}_m - \boldsymbol{a}_b]_\times \delta\boldsymbol{\theta} - \boldsymbol{R}(\delta\boldsymbol{a}_b + \delta\boldsymbol{g})\Delta t + \boldsymbol{v}_i \Big), \tag{10}$$

$$\delta\boldsymbol{\theta} \leftarrow \boldsymbol{R}^\top \Big\{(\boldsymbol{\omega}_m - \boldsymbol{\omega}_b)\Delta t\Big\}\delta\boldsymbol{\theta} - \delta\boldsymbol{\omega}_b\Delta t + \boldsymbol{\theta}_i, \tag{11}$$

$$\delta\boldsymbol{a}_b \leftarrow \delta\boldsymbol{a}_b + \boldsymbol{a}_i, \tag{12}$$

$$\delta\boldsymbol{\omega}_b \leftarrow \delta\boldsymbol{\omega}_b + \boldsymbol{\omega}_i, \tag{13}$$

$$\delta\boldsymbol{g} \leftarrow \delta\boldsymbol{g}. \tag{14}$$

Where $R\{\boldsymbol{\theta}\}$ is the rotation matrix computed from the angle vector $\boldsymbol{\theta}$ using Rodrigues' Formula. Finally, the injection step can be simply defined by

$$\boldsymbol{x} \leftarrow \boldsymbol{x} \oplus \hat{\delta\boldsymbol{x}} \tag{15}$$

Where $\bigoplus$ denotes regular addition apart from the operation with quaternions, which is given by eq.16.

$$\boldsymbol{q} \leftarrow \boldsymbol{q} \otimes \boldsymbol{q}\{\hat{\delta\boldsymbol{\theta}}\} \tag{16}$$

3.2.2 Covariance: Predict Phase

Having predicted the mean (or state), the predicted covariance matrix is given by 17

$$P \leftarrow F_x P F_x^\top + F_i R_i F_i^\top \tag{17}$$

Where F_i and F_x are the Jacobians with respect to the error and perturbation vectors, respectively.

R_i is the covariance matrix of the perturbation impulses. F_x can be further expanded to:

$$F_x = \frac{\partial f}{\partial \delta x}\Big|_{x,u_m} = \begin{bmatrix} I & I\Delta t & 0 & \\ 0 & I & -R'\{a_m - a_s]_x \Delta t & \boldsymbol{M_{9\times 9}} \\ 0 & 0 & R^T\{\omega_m - \omega_b \Delta t\} & \\ & \mathbf{0_9} & & \boldsymbol{I_9} \end{bmatrix}. \quad (18)$$

Where

$$\boldsymbol{M} = \begin{bmatrix} 0 & 0 & 0 \\ -R\delta t & 0 & I\delta t \\ 0 & -I\delta t & 0 \end{bmatrix}, \quad (19)$$

As for the Jacobian with respect to the perturbation vectors:

$$F_i = \frac{\partial f}{\partial u}\Big|_{x,u_m} = \begin{bmatrix} \mathbf{0_{3\times 12}} \\ \boldsymbol{I_{12}} \\ \mathbf{0_{3\times 12}} \end{bmatrix}, \quad (20)$$

Finally, the process noise covariance matrix is presented below:

$$\boldsymbol{R_i} = \begin{bmatrix} \boldsymbol{V_i} & 0 & 0 & 0 \\ 0 & \boldsymbol{\Theta_i} & 0 & 0 \\ 0 & 0 & \boldsymbol{A_i} & 0 \\ 0 & 0 & 0 & \boldsymbol{\Omega_i} \end{bmatrix} \quad (21)$$

Where $\boldsymbol{V_i}, \boldsymbol{\Theta_i}, \boldsymbol{A_i}$ and $\boldsymbol{\Omega_i}$ are the covariance matrices of each random perturbation impulses. These covariance matrices are calculated based on four parameters given in the IMU calibration: gyroscope and accelerometer "white noise" and "random walk", as described in [7].

3.2.3 Covariance: Update Phase

It is now possible to explicitly define our filter correction equations, which will allow the filter to estimate the error state:

$$\boldsymbol{K} = \boldsymbol{P}\boldsymbol{H}^\top(\boldsymbol{H}\boldsymbol{P}\boldsymbol{H}^\top + \boldsymbol{V})^{-1} \quad (22)$$

$$\hat{\delta \mathbf{x}} \leftarrow \boldsymbol{K}\left(\boldsymbol{y} - h(\hat{\mathbf{x}}_t)\right) \quad (23)$$

$$\boldsymbol{P} \leftarrow (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P} \quad (24)$$

The Jacobian $\mathbf{H}$ is the derivative of the observation model h with respect to the error state $\boldsymbol{\delta x}$, evaluated at the nominal state $\boldsymbol{x}$ and can be computed using the chain rule:

$$\mathbf{H} \triangleq \frac{\partial \mathbf{h}}{\partial \delta \mathbf{x}}\Big|_{\mathbf{x}} = \frac{\partial \mathbf{h}}{\partial \mathbf{x}}\Big|_{\mathbf{x}} \frac{\partial \mathbf{x}}{\partial \delta \mathbf{x}}\Big|_{\mathbf{x}} = \mathbf{H}_x\, \mathbf{X}_{\delta x}. \quad (25)$$

As for the first term, it corresponds to the Jacobian of our measurement function with respect to the best true-state estimate:

$$\mathbf{H}_x \triangleq \frac{\partial \mathbf{h}}{\partial \mathbf{x}}\Big|_{\mathbf{x}}. \quad (26)$$

it depends on the measurement function and will thus be detailed further below.

The second term, $\mathbf{X}_{\delta x}$, can be derived as explained in [3], and eventually takes the form:

$$\mathbf{X}_{\delta x} \triangleq \frac{\partial \mathbf{x}_t}{\partial \delta \mathbf{x}}\Big|_{\mathbf{x}} = \begin{bmatrix} \mathbf{I}_6 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}_{\delta\theta} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_9 \end{bmatrix}, \quad (27)$$

where $\mathbf{I}_x$ are identity matrices of dimension x, and $\mathbf{Q}_{\delta\theta}$ is the quaternion term, which can be simplified to:

$$\mathbf{Q}_{\delta\theta} = \frac{1}{2}\begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & -q_z & q_y \\ q_z & q_w & -q_x \\ -q_y & q_x & q_w \end{bmatrix}. \quad (28)$$

Last but not least, expanding the observation model:

$$\mathbf{y} = \mathbf{h}(\mathbf{x}_t, M_j) + \boldsymbol{\delta}_t \quad (29)$$

Where $\boldsymbol{\delta}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{V})$ is noise drawn from a white Gaussian distribution with covariance $\mathbf{V}$.

3.3 Implementation

This filter was implemented on MATLAB, mainly due to its simplicity and the absence of strict requirements concerning computational time. In the following section, we outline additional decisions made regarding the ES-EKF implementation.

3.3.1 ES-EKF choices

Measurement Function

The measurement model for the system is given by

$$\mathbf{h}(\mathbf{x}_t, M_j) = \begin{bmatrix} M_{j,x} - p_{t,x} \\ M_{j,y} - p_{t,y} \\ M_{j,z} - p_{t,z} \\ q_{t,w} \\ q_{t,x} \\ q_{t,y} \\ q_{t,z} \end{bmatrix}, \quad (30)$$

where $M_j = [M_{j,x}, M_{j,y}, M_{j,z}]^T$ represents the 3D position of the j-th landmark, $\boldsymbol{p_t}$ the UAV position and $\boldsymbol{q_t}$ its corresponding quaternion.

It's worth noting that the landmark's positions were selected to correspond to evenly spaced positions through the ground truth trajectory. This facilitated the creation of landmarks regardless of their number and the approach (simulated dataset *vs* true dataset). The first three elements of $\mathbf{h}(\mathbf{x}_t, M_j)_{1:3}$ thus correspond to the displacement between the landmark and the drone position.

The observation function is designed with simplicity to improve the quality of the results. Given the high-dimensional nature of the problem and its complex, non-linear dynamics, our approach aims to maximize the likelihood of achieving acceptable

convergence with the real dataset. This means that the advantage of the ES-EKF regarding linearization will be reflected only in the predicted step.

Outlier Detection
Our implementation includes outlier detection, *ie*, and the update step includes an association phase. As we were mainly concerned with the results' accuracy, we opted for a **batch** update. Since this topic was discussed in one of the laboratories, it will not be detailed extensively here. The central idea is to calculate the outlier likelihood for each observation, consistently considering all landmarks. In comparison to the sequential update, it requires more computational effort but it's less prone to errors in the outlier detection process.

Additionally, the threshold was defined based on the likelihood of each measurement rather than the Mahalanobis distance. This decision aimed at simplifying the calculations but an alternative approach could have been used.

3.3.2 Simulation & Validation Framework

The ES-EKF validation consisted of three phases:

1. Dynamic Model Validation:
2. Correction Steps Validation
3. Global Validation

In the first phase, we tested the dynamic model only. For that, the IMU measurements were inserted in the prediction function and the results were compared to the ground truth. While differences are expected, this approach enables a rough evaluation of the prediction step. However, this phase won't be presented as it's out of the scope of this section.

In the second phase, the filter accuracy was evaluated, with emphasis on the update step. While the ES-EKF results might diverge from the ground truth for several reasons (faulty prediction model, inaccurate update step, poor ground truth, etc), creating our own dataset using the aforementioned dynamic model enables us to focus on this step. With this in mind, four scenarios were devised:

Dataset	Noise Lv.	Outliers	Out. Det.
1	Low	No	NA
2	High	No	NA
3	Low	Yes	Turned Off
4	Low	Yes	Turned On

Table 1: Specifications of the four datasets based on noise level, presence of outliers, and outlier detection settings.

In each simulated dataset, noise was injected into both the IMU and landmark measurements beforehand. For the outliers, we defined a 50% rate, *i.e.*, at 50% of the time steps, all measurements were injected with a large, random noise vector.

It's worth noting that the number of landmarks, the outlier rate, the likelihood threshold and the injected noise levels were all defined as variables in our code. Therefore, anyone testing the program could easily modify these parameters in the `MAIN` function of the code.

In the third and final phase, the filter was completely tested with the real ground truth and IMU measurements. Different combinations of noise covariance matrices were be tested in order to reach the best possible results.

4 Experimental Results

In this section, we will be analyzing results obtained from both simulated and real-world data.

4.1 Simulated data

In the simulated data, pseudo-covariance matrices had to be introduced to simulate the noise influence on the measurements. Furthermore, it's relevant to say that all simulations occurred with a time step of $dt = 0.02$s.

4.1.1 Dataset 1 (Low Noise)

Dataset Explanation
In this first dataset, the noise levels injected into the system were considerably low. In the code, pseudo-covariance matrices $\tilde{R}$ and $\tilde{Q}$ (process and measurement, respectively) were defined to properly proceed with noise injection using the `mvnrnd` MATLAB function.

$$\tilde{R}_{6\times6} = \text{diag}(10^{-4}, ..., 10^{-4})$$
$$\tilde{Q}_{7\times7} = \text{diag}(10^{-4}, ..., 10^{-4})$$

As for the actual covariance matrices used in the ES-EKF, we defined

$$R_{12\times12} = \text{diag}(10^{-3}, ..., 10^{-3})$$
$$Q_{7\times7} = \text{diag}(10^{-3}, ..., 10^{-3})$$

Note that here the R matrix has different dimensions because the noise is assumed to influence four states of the error state vector, totalizing 12 components. It's worth pointing out that these values were also used throughout the simulations for datasets 3 and 4.

One of the advantages of the ES-EKF is the ability to deal with error accumulation. This means one can take the prediction step without

the update step and still get fairly good results. In this dataset, we took advantage of that and performed the update step every 0.1 seconds.

Results
The mean absolute error and the mean error are presented below for the position and quaternion state elements.

Position	ME	MAE
x	0.2769	3.775
y	-0.2238	3.671
z	-0.2368	3.550

(a) Position Errors (mm)

Quaternion	ME	MAE
q_w	0.5364	3.319
q_x	-0.0448	2.835
q_y	0.2417	2.523
q_z	0.0288	3.397

(b) Quaternion Errors (scaled by 10^3)

Table 2: Mean Error (ME) and Mean Absolute Error (MAE) for positions and quaternions

The errors are bounded within the intervals [-0.01; 0.01] for position and [-0.005; 0.005] for quaternions, which can be considered a good demonstration of stable behaviour.

Therefore, from Table 2 and from the error plots presented in A.1, one can conclude the ES-EKF yields reliable results.

4.1.2 Dataset 2 (High Noise)

Dataset Explanation
Then both the pseudo and the actual noise covariance matrices increased as follows:

$$\tilde{R}_{6\times6} = \text{diag}(1, ..., 1)$$
$$\tilde{Q}_{7\times7} = \text{diag}(1, ..., 1)$$
$$R_{12\times12} = \text{diag}(0.5, ..., 0.5)$$
$$Q_{7\times7} = \text{diag}(0.5, ..., 0.5)$$

The noise variance in the actual covariance matrices was intentionally set lower than that in the pseudo covariance matrices to assess its impact on the system.

In addition, the error accumulation approach was maintained: the simulation has a discrete time interval of 0.02s, while the update step is made every 0.1 s.

Results
Once again, the ME and MAE are presented in a table (Table 3). As expected, even (partially) accounting for the increase in the noise levels, the errors significantly increased. This is normal as there's no system comprehensively bulletproof to noise. The error plots A.2 further validate this conclusion and the trajectory plot B shows in a more meaningful and visual way the impact of higher noise levels.

Position	ME	MAE
x	0.02594	0.335
y	-0.04472	0.3309
z	0.03506	0.3672

(a) Position Errors (m)

Quaternion	ME	MAE
q_w	0.05229	0.2969
q_x	-0.02237	0.2503
q_y	-0.1449	0.2461
q_z	0.1128	0.2776

(b) Quaternion Errors

Table 3: Mean Error (ME) and Mean Absolute Error (MAE) for positions and quaternions.

4.1.3 Dataset 3 (No outlier detection)

Dataset Explanation
In this dataset, the previously mentioned 50% outlier rate was introduced and the noise levels were set to the initial values. The code was designed so that observation outliers would appear in evenly spaced time instants, although a random generation would also be a feasible approach.

From this dataset forth, the error accumulation approach was changed: the update step was always performed along the prediction step, *ie*, every 0.02 seconds—this decision aimed at improving the results accuracy, since the existence of outliers could heavily jeopardize the results, if not handled correctly.

Results
Table 4 shows once again the ME and MAE for the position and quaternion.
From A.3, one can easily conclude that the observation outliers heavily drifted the system from the true state, and since no correction accounting for the outliers was performed, the error increased significantly compared to dataset 1.

It's interesting to note that even in the absence of outlier detection, the system seems to attempt state correction, which can be inferred by the oscillations in the position and quaternions state elements. As expected, the error does not abruptly ramp up, as 50% of the observations are still inliers.

4.1.4 Dataset 4 (With outlier detection)

Dataset Explanation
In this last simulated dataset, the noise level was

Position	ME	MAE
x	-79.03	79.03
y	-74.39	74.39
z	18.4	18.4

(a) Position Errors (m)

Quaternion	ME	MAE
q_w	0.305	19.96
q_x	-0.556	10.84
q_y	-2.06	14.33
q_z	0.343	14.80

(b) Quaternion Errors

Table 4: Mean Error (ME) and Mean Absolute Error (MAE) for positions and quaternions

again kept small and the outlier detection (*ie*, the association step) was turned on. We defined a threshold on the outlier likelihood $\lambda = 0.1$, which was established based on some simulations run beforehand.

Results
In Table 5, the ME and MAE for the position and quaternion are shown.

Position	ME	MAE
x	-0.0138	1.856
y	-2.678e-04	1.821
z	0.0214	1.857

(a) Position Errors (mm)

Quaternion	ME	MAE
q_w	0.086	2.656
q_x	-2.79e-03	2.446
q_y	0.0395	2.112
q_z	0.1336	2.658

(b) Quaternion Errors (scaled by 10^3)

Table 5: Mean Error (ME) and Mean Absolute Error (MAE) for positions and quaternions

The errors are now virtually neglectable and even smaller than the ones from dataset 1. This could surprise a distracted reader, however one should note that the frequency of the update step was increased in comparison to the former dataset (from 0.1 s to 0.02s). As the observation outliers were evenly introduced time-wise, a 50% rate results in a system without outliers and a time interval between update steps of 0.04 s.

Surely this conclusion assumes that there were no errors in the association step but that was precisely the case, as it was confirmed by the simulation results. The error plots can be seen in A.4.

4.2 Real-world data

Dataset Explanation
Similarly to the previous datasets, the covariance matrices had to be defined beforehand. However, the pseudo-covariance matrix $\tilde{R}$ was not defined as the IMU measurements inherently already have noise. As for the update step measurements, they had to be obtained from the ground-truth, which justifies the existence of a corresponding pseudo covariance matrix. Hence, after running some simulations, these matrices were defined as follows:

$$\tilde{Q}_{7\times7} = \text{diag}(10^{-4}, ..., 10^{-4})$$
$$Q_{7\times7} = \text{diag}(10^{-3}, ..., 10^{-3})$$
$$R_{12\times12} = \text{diag}(5 \cdot 10^{-4}, ..., 5 \cdot 10^{-4})$$

The update and prediction steps always occurred simultaneously and the likelihood threshold was $\lambda = 0.1$, even though no outliers were explicitly introduced in the measurements (which will be explained later on).

Results
As shown in the **left** part of Table 6 and in A.5, the errors have significantly grown, even though the noise was kept small and no outliers were introduced.

Position	ME	MAE	ME	MAE
x	-0.2352	0.2432	-0.008518	0.01957
y	0.03394	0.1329	-0.001235	0.01265
z	0.03044	0.04171	0.002486	0.01223

(a) Position Errors (m)

Quaternion	ME	MAE)	ME	MAE
q_w	-0.3207	0.4330	-0.3031	0.4322
q_x	-0.3707	0.5085	-0.3941	0.5249
q_y	0.6793	0.7163	0.6637	0.7142
q_z	-0.1963	0.3474	-0.1747	0.3537

(b) Quaternion Errors

Table 6: ME and MAE for positions and quaternions from two simulations.

This was partially expected: when working with real-life obtained datasets, the measurement tools (IMU, GPS, etc) often yield values corrupted by noise that is hard to model. Besides not knowing the true variance, the noise might not be Gaussian. Therefore, not only the IMU measurements but also the ground-truth data may not correspond to the true state.

However, when looking at the position error plot (A.5), one may notice that, in some time intervals, the error ramps up without being corrected. After a brief analysis, it was concluded that the rejection resulting from the observations was due to the likelihood of those observations being virtually zero, hence being smaller than the threshold. When the

threshold was set to zero (to accept every observation), the results actually worsened. As no outliers were introduced, this was a bit unexpected.

One possible explanation might reside in the accumulated drift: as the errors keep growing, the likelihood for each observation will be smaller (since the innovation increases) and, at some point, the observations are rejected. However, as the path somewhat repeats itself (see B), the UAV may later approach a position close to the predicted one, so the observation is finally accepted.

Another way to interpret this is assuming that the measurement process is somewhat faulty (*e.g* misaligned predicted and actual measurement), which unfortunately was not identified. Therefore, the measurement noise variance (Q) was increased to 1 in an attempt to make the system less dependent on the measurements, which yielded the results in the right part of Table 6. Analyzing both the table and the error plots (A.6), it's crystal clear that the results improved significantly.

5 Summary and Conclusions

This report presented the workflow for implementing an ES-EKF in the context of UAV pose estimation. A 6-DoF was used to test the efficiency of the Kalman Filter in higher dimensional problems. The existing differences from the EKF were mentioned and the algorithm was explained. The use of quaternions aimed at avoiding singularity problems (Gimbal lock).

By using a craft-made dataset, the accuracy of the update step was verified. The mean absolute errors (MAE) and the error plots confirmed that in this situation, the ES-EKF was reliable, as long as the noise and the outliers were handled accordingly. However, the real dataset initially yielded unexpected results, with a MEA considerably higher. This might have suggested that the dynamic model was poorly implemented (*e.g.* approximation of constancy in acceleration during dt) but varying the measurement covariance showed that the problem resided in the measurement process. After fine-tuning the measurement noise variance, better results were obtained.

Even so, the final error wasn't totally neglectable. There are several possible explanations to this. Starting with the time step size, which may be larger than recommended. Besides, the impulse noise vectors may not completely reflect the perturbations in the IMU sensor. Furthermore, one has to always consider the uncertainties inherent to the ground truth data, since it was measured with real sensors.

Therefore, in future works, one might consider the implementation of dynamic noise covariance matrices for instance. This would make the filter adaptable to the uncertainties of the environment, both in the prediction and update steps. On a last note, it's worth saying that the implementation could have been further improved by analyzing the bias state elements (a_b, ω_b), introducing more random outliers and assessing the influence of non-Gaussian noise.

References

[1] Jeffrey Delmerico, Titus Cieslewski, Henri Rebecq, Matthias Faessler, and Davide Scaramuzza. Are we ready for autonomous drone racing? the UZH-FPV drone racing dataset. In *IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019.

[2] Raed Abu Zitar, Amani Mohsen, Amal Elfallah Seghrouchni, Frederic Barbaresco, and Nidal A. Al-Dmour. Intensive review of drones detection and tracking: Linear kalman filter versus nonlinear regression, an analysis case. *Archives of Computational Methods in Engineering*, 2023.

[3] Joan Solà. Quaternion kinematics for the error-state kalman filter, 2017.

[4] Marion Pilté, Silvère Bonnabel, and Frédéric Barbaresco. Drone tracking using an innovative ukf. In *Proceedings of the 3rd Conference on Geometric Science of Information (GSI 2017)*, 2017.

[5] Mohammed Abdulhakim Al-Absi, Rui Fu, Ki-Hwan Kim, Young-Sil Lee, Ahmed Abdulhakim Al-Absi, and Hoon-Jae Lee. Tracking unmanned aerial vehicles based on the kalman filter considering uncertainty and error aware. *Electronics*, 10(24):3067, 2021.

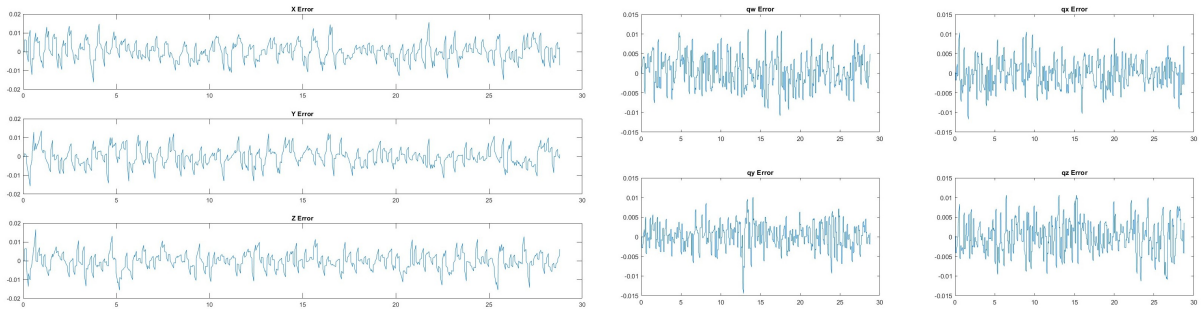
[6] Lovro Marković, Marin Kovač, Robert Milijaš, Marko Car, and Stjepan Bogdan. Error state extended kalman filter multi-sensor fusion for unmanned aerial vehicle localization in gps and magnetometer denied indoor environments. In *2022 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 983–990, 2022.

[7] ETH Zurich Autonomous Systems Lab (ASL). Kalibr imu noise model. `https://github.com/ethz-asl/kalibr/wiki/IMU-Noise-Model`. Accessed: 2025-01-04.

Appendix

A Error Plots

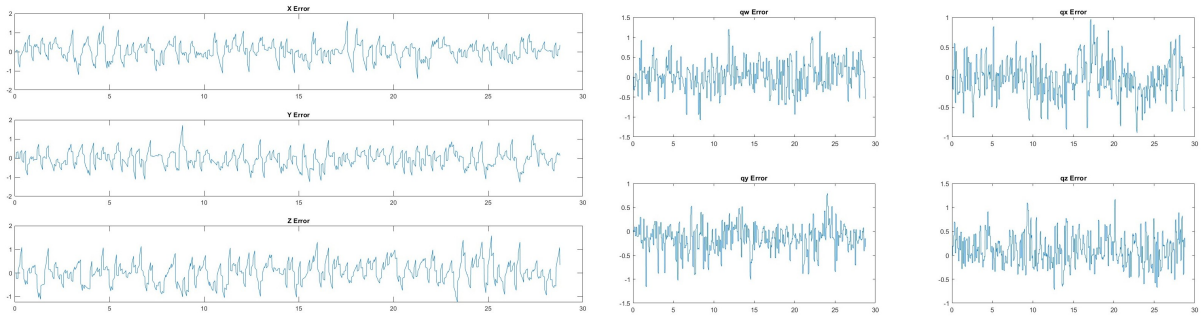
A.1 First Simulated Dataset



(a) Position (b) Quaternion

Figure 2: Error plot *vs* time

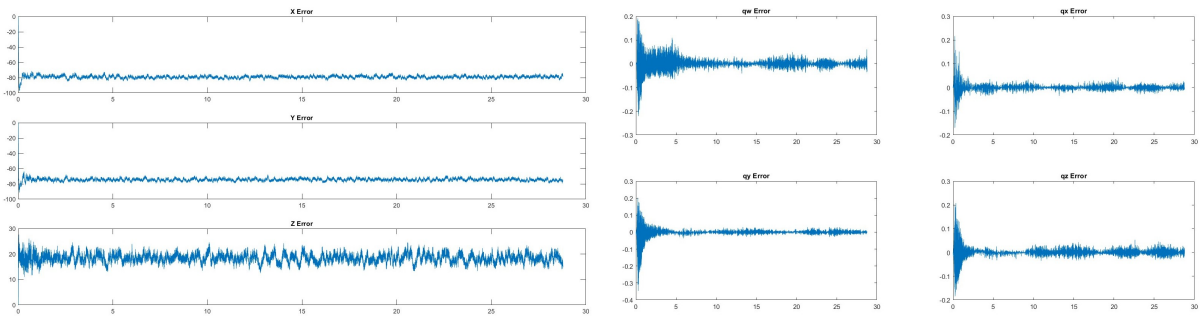
A.2 Second Simulated Dataset



(a) Position (b) Quaternion

Figure 3: Error plot *vs* time

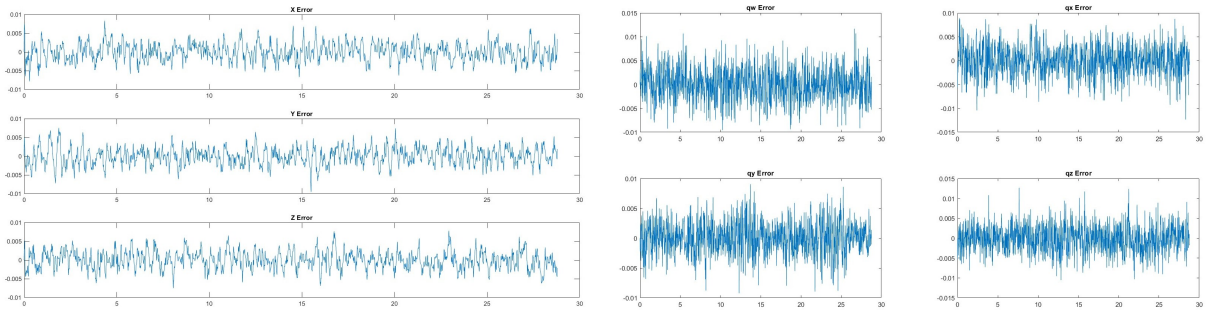
A.3 Third Simulated Dataset



(a) Position (b) Quaternion

Figure 4: Error plot *vs* time

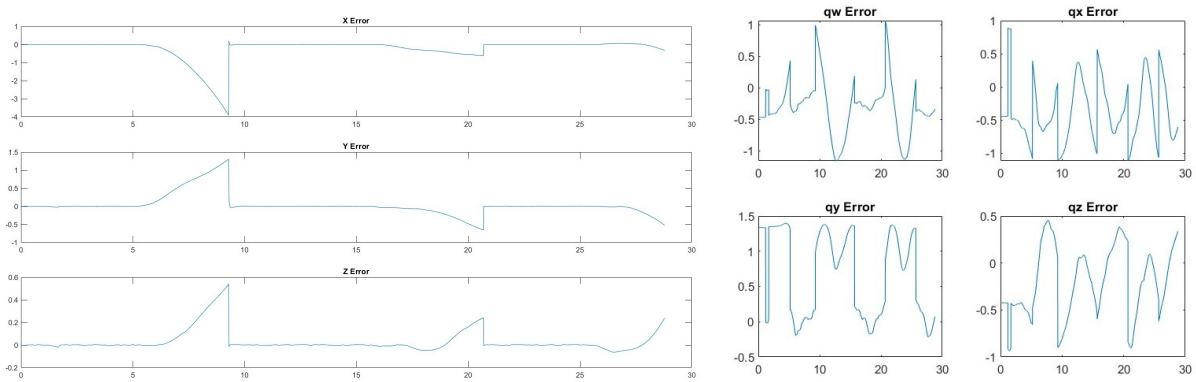
A.4 Fourth Simulated Dataset



(a) Dataset 1 (b) Dataset 2

Figure 5: Error plot *vs* time

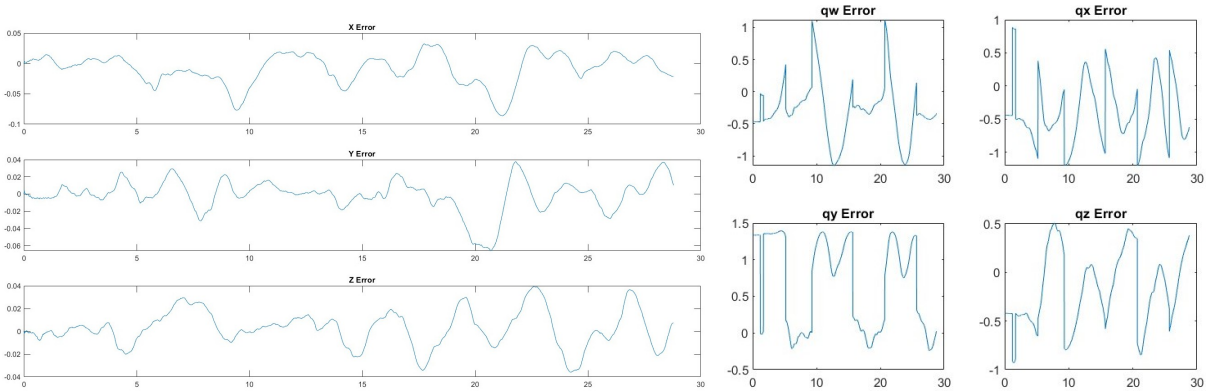
A.5 Real Dataset: First Simulation



(a) Dataset 1 (b) Real Dataset

Figure 6: Error plot *vs* time

A.6 Real Dataset: Second Simulation

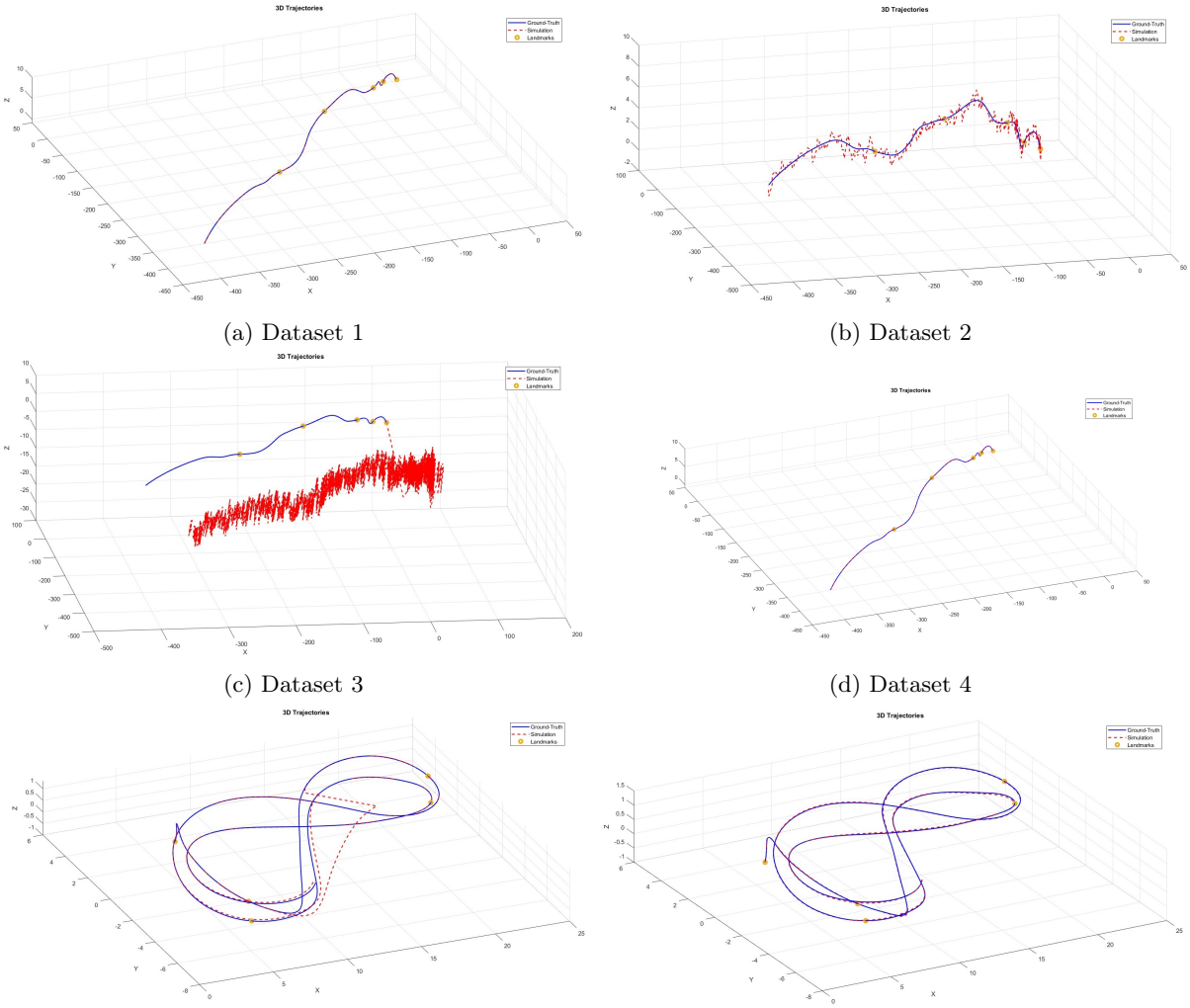


(a) Dataset 1 (b) Real Dataset

Figure 7: Error plot *vs* time

B Trajectory Plots

Trajectory plots for the simulated datasets: the ground truth trajectory (blue) and the actual trajectory (red). The landmarks are presented by orange circles.



(a) Dataset 1

(b) Dataset 2

(c) Dataset 3

(d) Dataset 4

(e) Dataset 4

(f) Dataset 4

Figure 8: Trajectories of the simulated UAV using different datasets/filter parameters.